Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences

Prof. Dr. Sascha Alda
Moritz Balg, Javed Razzaq, Luca Ringhausen und Team

6. Oktober 2026

Übungen zur Vorlesung Software Engineering I
WS 2026 / 2027

Übungsblatt Nr. 2
(Abgabe bis: Mittwoch, den 14. Oktober 2026, **09:00 Uhr**)

Aufgabe 1 (Modellierung eines Prozesses zur Abnahme, 15 Punkte):

Die Abnahme (engl.: *sign-off*) von Dokumenten ist insbesondere bei großen Software-Projekten, die nach den Vorgaben eines Wasserfall-Modells (vgl. Kapitel 2, ggf. vorarbeiten) entwickelt werden, ein oftmals unterschätzt komplexer Prozess. Einer offiziellen Abnahme geht in der Regel ein Kontrollprozess (engl.: *review*) voraus, in dem die *beteiligten* Stakeholder die Dokumente Korrektur lesen und ihre Kritiken äußern können.

Die Firma Modern Software GmbH wünscht sich von ihnen eine ausführliche Modellierung eines Prozesses einer Anforderungsspezifikation, das üblicherweise als Ergebnis der Phase „Anforderungsentwicklung“ in der Firma erstellt wird. Nehmen sie für Ihr Prozessmodell folgende Anforderungen und Überlegungen mit in Betracht, die von der Firma gestellt wurden:

- Die Anforderungsspezifikation wird von einem Business-Analysten erstellt. Nach Fertigstellung übergibt dieser die Dokumente an das PMO.

- Das Project Management Office (PMO) ist verantwortlich, nach Erhalt von den zu kontrollierenden Dokumenten durch den Business-Analysten, diese zunächst unter Anwendung des internen LLMs formal zu verifizieren und in einem *weiteren* Schritt diese auf einem internen Projekt-Cloud-Tool abzulegen. Anschließend werden die betreffenden Stakeholder benachrichtigt, die an einem Review beteiligt sind.

- An einem Review seien folgende Stakeholder beteiligt, welche *parallel* ein Review durchführen: der Projektleiter, der Kunde sowie der IT-Leiter. Definieren Sie mindestens drei sequentielle Aktivitäten pro Stakeholder, die hier in einem Review spezifisch durchgeführt werden. Seien Sie *kreativ* und versetzen Sie sich in die Rolle der jeweiligen Stakeholder!

- Jeder Stakeholder übergibt *nach* seinem Review seine Anmerkungen an das PMO. Erst wenn *alle* Bewertungen eingetroffen sind, bereitet das PMO, innerhalb *einer* Aktivität, die Reviews auf und führt eine finale Gesamtbewertung aus.

- Fügen Sie eine Fallunterscheidung ein, falls in der Anforderungsspezifikation während der Reviews Fehler entdeckt wurden. Beachten sie dabei die Fallunterschei-

dung bzgl. kritischer und wenig kritischer Fehler. Bei dem Vorhandensein von *kritischen* Fehlern soll, nach einer entsprechenden Überarbeitung der Anforderungsspezifikation durch den Business-Analysen, der Kontrollprozess erneut durchgeführt werden, also mit der erneuten Ausführung der Aktivität zur Erstellung einer Anforderungsspezifikation. Bei *wenig kritischen* Fehlern erfolgt eine direkte Überarbeitung der Dokumente, aber ohne Rücksprung in den Prozess. Anschließend kann eine Abnahme *ohne* Vorbehalt offener Punkte durch das PMO vorbereitet werden. Diese Form der Abnahme wird auch vorbereitet, wenn *keine* Fehler vorliegen.

- Beachten Sie auch den Fall, falls aus zeitlichen Gründen gemäß Projektplan die Abnahme des Dokuments dringend erfolgen muss, es jedoch immer noch kritische Fehler in der Spezifikation gibt. In diesem Fall sollten die kritischen Punkte dokumentiert werden und die Abnahme unter Vorbehalt offener Punkte durch das PMO vorbereitet werden.

- Unabhängig von der Form der Abnahme müssen dann die Stakeholder Kunde und IT-Leiter einen zuvor durch das PMO ausgegebenen „Letter of Acceptance“ (inklusive Abnahmeprotokoll) *parallel* unterschreiben. Erst wenn *alle* Unterschriften vorliegen, kann das PMO die nächste Phase („System-Design“) einleiten.

Skizzieren Sie ihr Prozessmodell zur Abnahme einer Anforderungsspezifikation mit Hilfe eines Aktivitätsdiagramms von der UML. Dokumente und Swimlanes brauchen sie in ihrem Modell keine zu berücksichtigen. Informationen zu diesem Diagrammtyp finden sie in dem Buch:

Rupp, Chris et al.: *UML 2 Glasklar – Praxiswissen für die UML-Modellierung*. Hanser Verlag, 2012 (5. Auflage).

Dieses Buch wird in dieser Vorlesung *ausschließlich* als Referenz verwendet! Bitte verwenden sie keine „Online“-Quellen (Wiki etc.)! Eine erste Einführung dazu findet in den Übungen statt.

Aufgabe 2 (Entwicklung und Test einer Container-Klasse, 15 Punkte)

Es soll ihre Aufgabe sein, eine Klasse `Container` zu implementieren, die zur Laufzeit verschiedene (mehrere) „Member-Objekte“ aufnehmen und *intern innerhalb* des `Container`-Objekts abspeichern kann. Diese „Member-Objekte“ implementieren das Interface `Member`.

Das Interface können sie über das GitHub-Repository der Vorlesung beziehen (bitte auch den Kommentar über das Interface beachten!):

https://git.fslab.de/salda2m/se1ws26

Hierzu die neuen Codes über einen Pull beziehen. Eine kleine Anleitung dazu finden Sie im vierten Teil des Tutoriums zur Integration von GitHub in IntelliJ:

https://www.youtube.com/watch?v=I4L0k33TNQ4

Falls es zu einer Fehlermeldung („Pull rejected“) beim Pull kommt, dann unter bitte unter „Modify Options“ die Option „rebase“ setzen. Bitte vorab auch Ihre Änderungen am Source Code committen (siehe Erläuterung in dem Video).

Die Klasse `Container` soll folgende (funktionale) Anforderungen (*FA*) erfüllen:

FA1: Es soll möglich sein, Objekte vom Typ `Member` in einem instanziierten Objekt der Klasse `Container` zur Laufzeit abzuspeichern. Um zur Laufzeit Objekte vom Typ `Member` zu erzeugen, sollten sie eine entsprechende Klasse (z.B. `ConcreteMember`) bereitstellen, welches das Interface `Member` implementiert. Bitte bei der Implementierung auch den Kommentar aus dem Interface beachten.

Eine Kontrolle, ob ein übergebenes `Member`-Objekt mit einer ID bereits in dem `Container`-Objekt enthalten ist, sollte ebenfalls in der `Container` -Klasse implementiert werden. Falls ein `Member`-Objekt mit einer bereits vorhandenen ID übergeben wird, so soll eine *geprüfte* Exception vom Typ `ContainerException` ausgeworfen werden. Die Klasse `ContainerException` bitte selbst implementieren! Die zugehörige Message dieser Exception sollte folgendes *exakt* ausgeben:

```
„Das Member-Objekt mit der ID [hier die ID des Objekts] ist bereits vorhanden!“
```

Spezifikation der Methode nach der UML:

```
+ addMember( member : Member ) : void {throws ContainerException}
// Bemerkung: streng genommen kennt die UML keine Exceptions
```

FA2: Es soll mit der Methode `deleteMember` möglich sein, Objekte vom Typ `Member` in einem instanziierten Objekt der Klasse `Container` zur Laufzeit zu löschen. Dazu muss die eindeutige ID des Objekts vom Typ `Member` dem `Container`-Objekt übergeben werden. Falls zu der übergebenen ID kein `Member`-Objekt gespeichert ist, sollte über einen von *ihnen freiwählbaren* Rückgabewert eine entsprechende Fehlermeldung ausgegeben werden. Welche Nachteile ergeben sich aus ihrer Sicht für ein solchen Fehlerhandling gegenüber einer Lösung mit Exceptions? Kurzes Statement!

Spezifikation der Methode nach der UML:

```
+ deleteMember( id : Integer ) : String
```

FA3: Es soll mit einer Methode `dump` möglich sein, die IDs der aktuell abgespeicherten Objekte vom Typ `Member` auszugeben (hier: direkt auf der Console.). Für die Ausgabe der ID eines einzelnen `Member`-Objekts soll innerhalb der Methode `dump` die Methode `toString()` des jeweiligen `Member`-Objekts verwendet werden, die dazu aus der Klasse `java.lang.Object` überschrieben werden muss. Die Ausgabe, die bei `toString()` erfolgt, sollte wie folgt für ein `Member`-Objekt aussehen:

„`Member (ID = [hier die ID des Members])`“

Spezifikation der Methode nach UML:

```
+ dump( ) : void
```

FA4: Es soll möglich sein, die Anzahl der aktuell abgespeicherten Objekte vom Typ `Member` in einem `Container`-Objekt mittels der Methode `size` zu ermitteln und an ein aufrufendes Objekt als Rückgabeparameter zu übergeben.

Spezifikation der Methode nach UML:

```
+ size( ) : int
```

1.)
Zur Umsetzung dieser Anforderungen *müssen* sie in der Klasse `Container` eine Listen-Datenstruktur wiederverwenden, um Objekte intern abzuspeichern. Recherchieren sie dazu in dem Package `java.util.*` nach brauchbaren Strukturen. Bitte *keine* eigene Listen-Implementierung realisieren ;-) Diese List-Datenstruktur muss intern als Objektvariable bereitgestellt werden. Die Datenstruktur `HashMap` darf *nicht* verwendet werden.

2.)
Testen sie die Implementierung der Klasse `Container` mit Hilfe einer externen Testklasse (JUnit5 kann verwendet werden, falls bekannt). Erzeugen sie dabei zwei verschiedene `Member` -Objekte und testen sie ihre Implementierung *hinreichend* mit mindestens 10 aufeinanderfolgenden Testfällen (hier: 10 Assertions in einer Test-Methode).

Hinweis dazu: formulieren sie ihre Testfälle zustandsbasiert auf Basis der Anzahl von intern gespeicherten `Member` –Objekten (= den Zustand des Container-Objekts), die beim Hinzufügen / Löschen von Objekten variieren kann (weitere Hinweise dazu in der Übung). Dokumentieren sie ihre Testfälle zusätzlich mit dem Excel-Template (LEA → Kapitel 1 → Material, Tab „Simple Test Suite“). Bitte auch Äquivalenzklassen definieren. Laden Sie auch das Zustandsmodell hoch.

3.)
Alle Klassen und Interfaces und mögliche Unterpackages ihrer Lösung sollen in einem *neuen* Package `org.hbrs.se1.ws26.uebung2` abgelegt werden.